Which AI System Should a Bond Desk Deploy to Read the Fed?

Evaluating a fine-tuned RoBERTa and two Gemini models on hawkish, dovish and neutral stance in FOMC sentences

Jaanvi Bajaj and Nikhil Balaji · DS602 Topics in Data Science for Economics · Mini-project, Lessons 5–6 · 30 September 2026

Summary. We compared three systems for tagging sentences from US Federal Reserve communications as hawkish, dovish or neutral: roberta-base fine-tuned on 1,984 labelled sentences (S1), Gemini 3.5 Flash-Lite (S2) and Gemini 3.5 Flash (S3), both used zero-shot. On a 496-sentence test set, S1 had the highest macro F1 (0.660) and was by far the fastest, but it swapped hawkish and dovish twice as often as the Gemini models. Once we priced that error for a trading desk, Gemini Flash and a RoBERTa-to-Flash cascade became the cheapest options. None of the pairwise differences was statistically significant after a Holm correction. When we relabelled 50 test sentences ourselves, we agreed with the dataset only 58% of the time, which suggests that part of what looks like model error is disagreement about the labels.

1. The business problem

The Federal Open Market Committee (FOMC) sets US interest rates and explains its decisions in statements, minutes and speeches. Traders read these documents closely, because bond prices move with expectations about future rates. A single sentence can lean **hawkish** (towards tighter policy and higher rates), **dovish** (towards easier policy and lower rates), or be **neutral**.

We framed the project around one decision: **which system should a bond desk deploy to tag every FOMC sentence automatically?** The tags would be combined into a stance signal that informs rate positioning. The point that shaped our whole evaluation is that errors do not cost the same. If a model reads a hawkish sentence as dovish, or the reverse, the desk may take a position in the wrong direction. We call this a **flip**. If a model calls a hawkish sentence neutral, the desk simply misses a signal. That is a lost opportunity, not a wrong-way trade. A useful system therefore has to be accurate, fast enough to use while a statement is being read, cheap to run, and above all it must avoid flips.

2. Data and test set

We used the public **gtfintechlab/fomc_communication** dataset from Hugging Face, which contains human-labelled sentences from FOMC documents. It has 1,984 training sentences and 496 test sentences. We used the training split only to fine-tune S1, so no test sentence was ever seen during training. The test set is imbalanced: 247 neutral (50%), 135 dovish (27%) and 114 hawkish (23%). While exploring the data we found that the year column is unreliable (sentences about COVID are tagged with years such as 2005), so we did not use it.

To test the systems beyond the clean test set, we built five additional sets, summarised in Table 1. Typos and hedges were generated by code. The 16 negations were drafted with the help of an AI assistant and every label was checked by us. The 14 fresh sentences come from the July and September 2026 FOMC statements, which were released after the benchmark was published, and we labelled them ourselves. We also relabelled 50 test sentences blind, without seeing the gold label, to measure how much humans agree with the dataset.

Table 1. Test sets used in the evaluation

Set	Content	Size	Purpose
Clean test	Benchmark test split with gold labels	496	Main comparison
Length	Shortest and longest 20% of test sentences by word count	105 / 110	Effect of input length
Adversarial	Typos (17), hedges (17), negations that reverse the meaning (16)	50	Sensitivity to wording
Multilingual	Test sentences translated into Hindi and Simplified Chinese	100	Non-English input
Out of domain	European Central Bank sentences from the same lab	100	A different central bank
Fresh 2026	Sentences from July and September 2026 FOMC statements	14	Text newer than the benchmark
Blind relabel	50 test sentences labelled by us without seeing gold	50	Human agreement with gold

3. Systems compared

We chose three systems that represent two different approaches. **S1** is roberta-base, a small language model that we fine-tuned for four epochs on the training split (learning rate 2e-5, AdamW, linear warm-up). We originally planned to use the published FOMC-RoBERTa model, but access to it is gated, so we trained our own. Validation macro F1 on a held-out 10% of the training data rose from 0.491 after the first epoch to 0.656 after the last. **S2** (Gemini 3.5 Flash-Lite) and **S3** (Gemini 3.5 Flash) are general-purpose language models from the same family at two sizes. Both received the same zero-shot prompt, which defines the three labels and asks for exactly one word, with temperature 0 and a 20-token output limit. Every reply was checked by a parser, and any answer that was not one of the three labels would count as a failure. There were none.

We also included two baselines: **B1**, which always predicts neutral, and **B2**, which predicts at random. One difference between S2 and S3 only appeared while running the code: Flash-Lite does not allow its internal reasoning (“thinking”) to be switched off, so S2 ran with thinking enabled, while S3 ran without it. We return to this in the results.

4. Metrics and decision rule

Because half of the test set is neutral, accuracy would reward a model that always says neutral: B1 reaches 0.498 accuracy. We therefore used **macro F1** as the main quality metric, since it gives the three classes equal weight. B1 scores only 0.222 on it. We reported macro F1 with a 95% bootstrap confidence interval, plus per-class precision and recall. For the desk we added the **flip rate** and a **business cost** per 1,000 sentences, in which a flip costs 5 points and a missed or false signal costs 1. The weight of 5 is our assumption, so we repeated the calculation with weights of 3 and 10. For efficiency we measured latency at the 50th, 95th and 99th percentiles, throughput, tokens per request and API cost. Finally, we checked the calibration of S1’s confidence, because a cascade only works if the model’s confidence is meaningful.

Before running the models, we wrote a decision rule into the notebook: *deploy the cheapest system that (a) has macro F1 within 2 points of the best system, (b) has 95th-percentile latency under one second per sentence, and (c) does not flip significantly more often than the best system; if none qualifies, deploy the system with the highest macro F1.* The rule is applied by code at the end of the notebook, so the recommendation does not depend on our judgement after seeing the numbers.

5. Evaluation pipeline

The evaluation is a single Colab notebook that runs from top to bottom. All settings (model names, prices, random seed 42, cost weights, sample sizes) sit in one configuration cell. Every system runs on every test sentence through one function that times each call and saves each answer to Google Drive. Re-running the notebook therefore reuses saved answers and costs nothing, which also protected us when Colab disconnected during a 24-minute run. A mock mode runs the whole pipeline on fake data in about a minute, which we used to test the code before paying for API calls. For statistics we used a paired bootstrap with 2,000 resamples, McNemar's test for each pair of systems, and a Holm correction for the three comparisons. We used AI coding assistants to help write parts of the pipeline, and we reviewed and ran every cell ourselves.

6. Results

6.1 Quality and statistical significance

Table 2. Results on the 496-sentence clean test set

System	Macro F1 [95% CI]	Accuracy	Flip rate	p95 latency	USD per 1,000	Business cost per 1,000
B1 majority	0.222 [0.207, 0.234]	0.498	0.000	–	–	502
B2 random	0.315 [0.275, 0.356]	0.329	0.194	–	–	1,446
S1 RoBERTa	**0.660** [0.616, 0.703]	**0.679**	0.062	**0.012 s**	0*	571
S2 Flash-Lite	0.595 [0.545, 0.643]	0.645	**0.028**	0.961 s	0.039**	468
S3 Flash	0.622 [0.573, 0.668]	0.651	**0.028**	1.975 s	0.191	462
S1→S3 cascade, t = 0.9	0.676†	–	**0.028**	1.741 s	0.111	**415**

** No API cost, but S1 needs a GPU, which is not included. ** Excludes Flash-Lite's thinking tokens, so the true cost is higher. † The cascade threshold was chosen on the test set, so this figure is optimistic.*

S1 has the highest macro F1, but its confidence interval overlaps with S3's (Figure 1). The paired comparison in Table 3 makes this precise. The bootstrap interval for the S1–S2 difference (0.009 to 0.123) excludes zero, but McNemar's test does not find the difference significant, and after the Holm correction no pair is significant. The two tests disagree because they ask different questions: McNemar compares how many sentences each system gets right, while the bootstrap compares macro F1. S1's advantage comes from being balanced across the three classes rather than from getting more sentences right. Per class, the Gemini models recognise neutral sentences well (recall 0.85 and 0.78) but miss many hawkish ones (recall 0.38 and 0.45), while S1's recall is between 0.62 and 0.73 for every class.

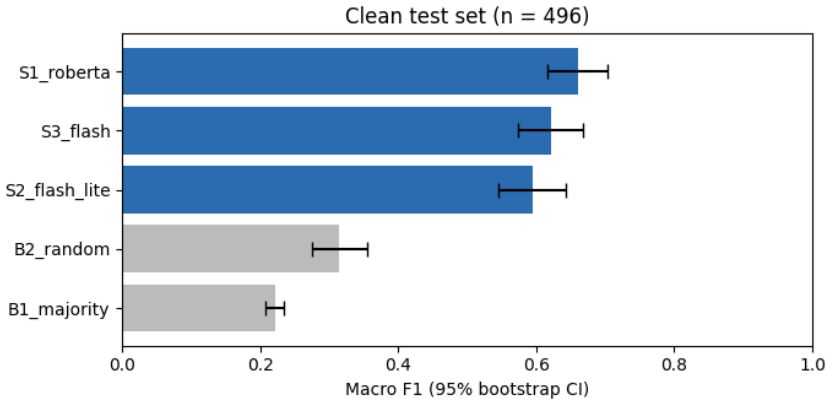


Figure 1. Macro F1 with 95% bootstrap confidence intervals (2,000 resamples). Grey bars are baselines.

Table 3. Pairwise comparisons

Pair	F1 difference	Paired 95% CI	Only first / only second right	McNemar p	Holm p
S1 vs S2	+0.066	[0.009, 0.123]	86 / 69	0.199	0.596
S1 vs S3	+0.039	[−0.012, 0.091]	75 / 61	0.265	0.596
S2 vs S3	−0.027	[−0.062, 0.009]	26 / 29	0.787	0.787

6.2 Speed, cost and the business view

S1 answers a sentence in 9 milliseconds at the median and 12 ms at the 95th percentile, roughly 85 times faster than Flash-Lite and 140 times faster than Flash at the median (Flash-Lite 0.76 s, Flash 1.28 s; 95th percentiles 0.96 s and 1.98 s). The Gemini latencies include network time from Colab. API costs are small in absolute terms: about 4 cents per 1,000 sentences for Flash-Lite and 19 cents for Flash.

The business cost changes the picture. The confusion matrices (Figure 2) show that S1 makes 31 flips, while each Gemini model makes 14. With a flip weighted five times a missed signal, S1 has the highest cost of the three systems, and is even more expensive than always predicting neutral. Table 4 shows that the ranking does not depend on the exact weight. Working backwards from the error counts, S1 would only be cheaper than Flash if a flip cost less than about 1.8 times a missed signal, which we do not think any trader would accept.

Table 4. Business cost per 1,000 sentences at three flip weights (lower is better)

System	Flip cost 3	Flip cost 5	Flip cost 10
S3 Flash	**405**	**462**	**603**
S2 Flash-Lite	411	468	609
B1 majority	502	502	502
S1 RoBERTa	446	571	883
B2 random	1,059	1,446	2,413

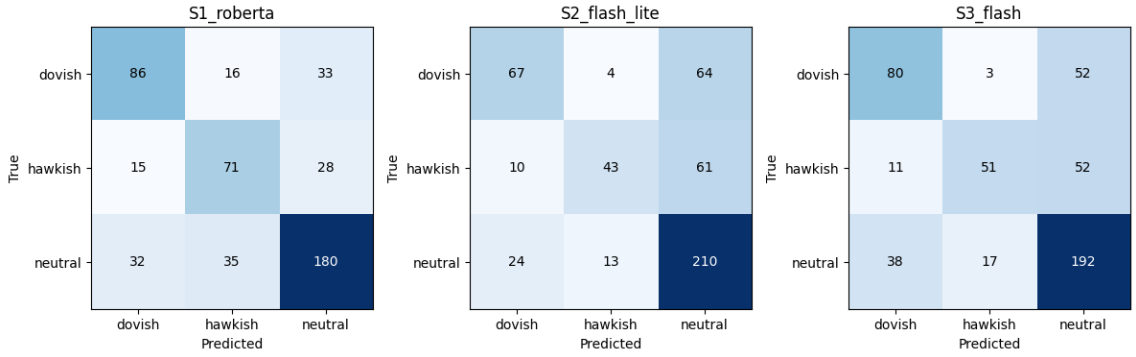


Figure 2. Confusion matrices on the clean test set. Rows are gold labels, columns are predictions. Flips appear in the dovish–hawkish corners.

6.3 Calibration and the cascade

S1's confidence is not well calibrated: its expected calibration error is 0.141, and in most confidence bins it is more confident than it is accurate. Its confidence still ranks sentences well, with an area under the ROC curve between 0.83 and 0.86 for each class. This led us to test a cascade: S1 answers first, and any sentence where S1's confidence is below a threshold is sent to Flash instead. Because we already had both systems' answers, testing this cost nothing. At a threshold of 0.9, 58% of sentences go to Flash, macro F1 rises to 0.676, the flip rate matches Gemini (0.028) and the business cost falls to 415, the lowest of all options. The weakness is that we chose 0.9 after looking at test results, so this score is optimistic, and its 95th-percentile latency (1.74 s) is above our one-second target.

6.4 Robustness and stability

Figure 3 shows macro F1 on each stress-test set. The clearest result is that S1 fails on Hindi and Chinese, where its macro F1 drops to 0.222, exactly the level of the always-neutral baseline. This is expected, since roberta-base was trained on English. On European Central Bank sentences S1 held its score (0.659), which we think reflects that the dataset comes from the same lab and uses the same labelling rules. Flash was the strongest on adversarial edits (0.803). The main weakness here is negation: S1 labelled only half of the negated sentences correctly, compared with 0.81 for Flash. Short sentences hurt Flash-Lite the most (0.467). On the 14 fresh 2026 sentences we used accuracy instead of macro F1, because the set contains no dovish sentence. All three systems scored between 0.57 and 0.64. S1 dropped as much as the Gemini models even though it cannot have seen this text, so we read the drop as a change in the Fed's wording rather than evidence that the LLMs had memorised the benchmark.

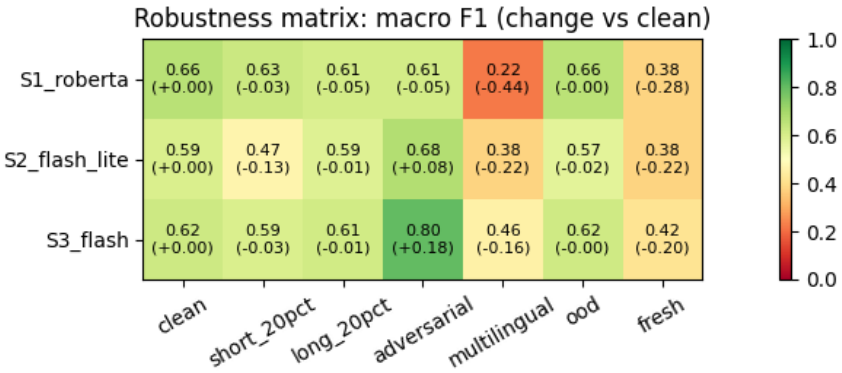


Figure 3. Robustness matrix: macro F1 on each set, with the change from the clean set in brackets.

We also ran both Gemini models three times on the full test set. Flash gave identical answers every time. Flash-Lite changed its answer on 6% of sentences, and its macro F1 ranged from 0.575 to 0.595, so the 0.595 in Table 2 is the top of its range. The most likely reason is that Flash-Lite ran with thinking enabled. This also means the two Gemini models were not configured identically, which is a limitation of our comparison.

6.5 Does the stance signal track real decisions?

As a final check we tagged eight real FOMC statements from October 2025 to September 2026 and computed a stance index for each: the share of hawkish sentences minus the share of dovish ones. We removed the sentences that announce the rate decision and the votes, because they would give the answer away. All three systems produced negative values before the two rate cuts in late 2025 and positive values in the 2026 tightening period. However, none of them rose before the September 2026 hike, because the June to September statements use almost identical wording. The index separates the easing and tightening periods but does not predict individual meetings. With eight points, we treat this as a sanity check only.

7. Error analysis

We analysed errors in three layers. First, we split every error on the clean set into types (Figure 4). S1 makes the fewest errors overall (159, compared with 173 for Flash and 176 for Flash-Lite), but more of them are flips. Most Gemini errors are missed signals: 125 for Flash-Lite and 104 for Flash. In other words, S1 commits to a direction and is sometimes wrong, while the Gemini models retreat to neutral when unsure.

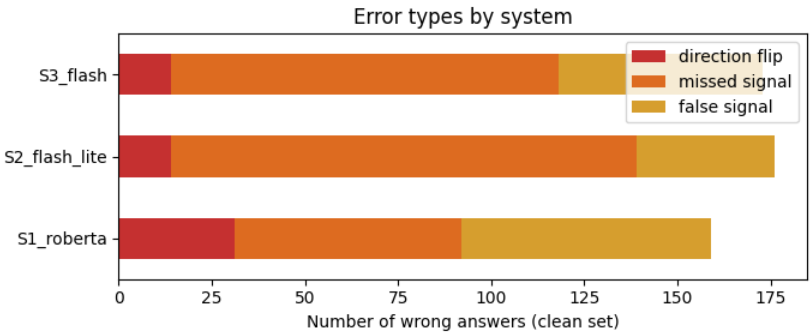


Figure 4. Error types by system on the clean test set.

Second, we tagged sentences by simple keyword patterns and compared error rates. Gemini struggled with conditional sentences containing “if” or “unless” (57% of these wrong for Flash-Lite and 52% for Flash, against 33% for S1), while S1 struggled most with hedged sentences such as “some participants noted” (41% wrong). These tags are rough and some groups are small, so we treat them as indications rather than findings.

Third, we read 30 errors ourselves, ten per system, and sorted each into one of four categories using rules we wrote beforehand (Table 5). Half of them, 15 of 30, turned out to be cases where we thought the gold label was debatable. A typical example is “The unemployment rate ticked down to 5”, which the dataset labels hawkish but which states a fact without any policy stance, so a neutral answer is reasonable. Only 7 of the 30 were clear model mistakes, such as S1 labelling “there did not seem to be any urgency to tighten current policy” as neutral when it is dovish. This fits with our blind relabelling: we agreed with the gold labels on only 58% of the 50 sentences (Cohen’s $\kappa = 0.31$), while Flash-Lite and Flash agreed with our labels 76% and 78% of the time. S1 agreed more with gold (70%) than with us (66%). Our interpretation is that S1 has learned the labelling conventions of this dataset, while the Gemini models read the sentences more like we did. Part of the gap between the systems is therefore a question of label convention rather than model quality.

Table 5. Thirty errors read by hand

System	Model wrong	Gold disputable	Genuinely ambiguous	Not policy text
S1 RoBERTa	3	4	0	3
S2 Flash-Lite	1	6	3	0
S3 Flash	3	5	1	1
Total (30)	**7**	**15**	**4**	**4**

8. Recommendation

Applying our decision rule, no system met all three conditions. S1 is fast and within the quality range but flips significantly more than the best system. Flash-Lite and Flash are more than two points below the best macro F1, and Flash is also too slow. The cascade has the best macro F1 but misses the latency target. The rule therefore falls back to the highest macro F1, which is **the S1-to-Flash cascade at a threshold of 0.9**.

We agree with this direction, with two conditions. The cascade’s threshold must be chosen again on a separate validation set, since our figure is optimistic, and its latency must be brought under the target, for example by sending sentences to Flash in parallel. Until that is done, if the desk had to go live immediately we would deploy Gemini Flash on its own: it has the lowest business cost at every weight we tested, the fewest flips, nothing in its setup was tuned on the test set, and it gave identical answers across repeated runs. S1 alone only makes sense if speed matters more than direction errors.

9. Business takeaways

The most accurate model is not the cheapest one to be wrong with. S1 wins on macro F1 and speed, but its flips make it the most expensive system for the desk. A standard accuracy comparison would have recommended the wrong system for this use.

How errors are priced matters more than which model is used. The ranking depends on the ratio between the cost of a flip and the cost of a missed signal. A desk should agree on that ratio before choosing a model.

A small model and a large model can work together. Letting the fast specialist handle confident cases and passing the rest to a stronger model gave the best quality and the lowest cost in our tests.

Language coverage is a real risk. A specialist trained on English text becomes useless on other languages, which matters for a desk that also follows non-US central banks.

The labels need attention. With 58% agreement between us and the dataset, part of any measured difference may reflect labelling choices. A desk building its own system should write clear labelling rules first.

10. Limitations

Our human evaluation relies on one annotator for 50 sentences and 30 errors, so we could not measure agreement between two of us. Several stress-test sets are small, in particular the 14 fresh and 16 negated sentences, and the negations were drafted with AI help. Flash-Lite ran with thinking enabled while Flash did not, and Flash-Lite's thinking tokens are not included in its cost. Latency was measured from Colab over the internet and would differ in production, and S1's speed assumes a GPU whose cost we did not include. The translations were produced by Flash, which may favour S3 on the multilingual set. Finally, the cascade threshold was selected on the test set, and the business cost weights are assumptions rather than measured losses.

11. What we learnt

The most useful lesson for us was that the choice of metric can decide the outcome. We started expecting to report one winner by macro F1, and ended up with three different winners depending on whether we looked at quality, speed or the cost of mistakes. Writing the decision rule before seeing results forced us to be clear about what the desk actually needs, and it stopped us from choosing a winner after the fact.

We also learnt to be careful about how much a small test set can show. Differences that looked large in a table, such as 0.660 against 0.622, were not significant once we used confidence intervals and corrected for multiple tests. Reading the errors ourselves was the most revealing step: it showed that many "errors" were sentences where we disagreed with the dataset too. On the practical side, saving every model answer as it arrived saved both time and money when Colab disconnected, and a temperature of 0 did not guarantee identical answers from an LLM. If we repeated the project, we would set aside a validation split from the start, use two annotators, and configure both Gemini models identically.

Reproducibility

The notebook fomc_stance_eval.ipynb runs end to end in Google Colab. All model answers, hand labels, figures and result tables (result_table.csv, pairwise_tests.csv, robustness_matrix.csv) are saved in the project folder, so the results in this report can be reproduced without repeating the API calls.